

Protótipo para detecção, localização e seleção automática de ação por sensoramento

Diagrama de Classes

Gabriel Vasconcellos de Moraes — Matheus Emanuel Gomes Souza

Orientador: Anker Douglas Loss

Curso: Engenharia da Computação — Faculdade do Centro Leste (UCL)

1 Apresentação

Este documento especifica o diagrama de classes do sistema **RadarTorres**, descrevendo os principais elementos de software (*Models*, *Services*, *Repositories* e *ViewModels*) e seus relacionamentos. Complementa o *Modelo de Banco de Dados* (persistência) e o *Diagrama de Implantação* (distribuição física), mostrando a estrutura estática do código-fonte em C# 13 / WPF (padrão MVVM), incluindo o uso pleno de Programação Orientada a Objetos: interfaces *I*Service/I*Repository* com implementação separada, herança de *INotifyPropertyChanged*, composição e generics.

Todas as classes, atributos e métodos listados a seguir foram lidos diretamente do código-fonte em `src/RadarTorres.App/`; nenhum nome foi inventado. Onde a fonte é uma inferência a partir do uso documentado (não leitura direta da assinatura completa), isso está marcado explicitamente na Seção 6.

2 Principais Elementos e Responsabilidades

2.1 Models (entidades de domínio)

Tabela 1: Classes de domínio (Models)

Classe	Responsabilidade
Target	Alvo detectado, vivo em memória, atualizado a cada leitura pelo mesmo Id (Angle, Distance, X, Y, Quadrant, IsActive, IsSelected, SelectedTower). Implementa <i>INotifyPropertyChanged</i> .
Tower	Torre demonstrativa carregada de <code>appsettings.json</code> (Id, Name, X, Y, PreferredQuadrant, State, IsAvailable). Implementa <i>INotifyPropertyChanged</i> .
SensorReading	DTO imutável de uma leitura já validada (TargetId, Angle, Distance, ReceivedAt, Source), consumido por <code>TargetTrackingService.ProcessReading</code> .
DeadZone	Zona onde alvos não recebem torre/acionamento (Id, Name, Type, Quadrant, MinDistance, MaxDistance, Enabled). Implementa <i>INotifyPropertyChanged</i> .
Usuario	Conta de acesso ao aplicativo (Id, Nome, Login, SenhaHash, SenhaSalt, Perfil, Ativo).
ObjetoDetectado	Registro histórico (CSV) de uma primeira detecção, gerado a partir do evento <code>TargetCreated</code> (Id, Tipo, X, Y, Z?, Quadrante, DataHora, Dispositivo).

Tabela 1 (continuação)

Classe	Responsabilidade
AcaoRealizada	Auditoria de acionamento (só inserção), gravada por FireControlService (Id, Dispositivo, TipoAcao, Origem, Resultado).
AlteracaoModo	Auditoria de troca de SystemMode (só inserção) (ModoAnterior, NovoModo, UsuarioSolicitante, Resultado).
PreferenciasUsuario	Preferências de UI por usuário, relação 1:1 com Usuario (UsuarioId como PK=FK, Idioma, Tema, SidebarRecolhida).
ChamadoAjuda	Chamado de suporte aberto pelo usuário (Id, UsuarioId, Titulo, Descricao, Status).
DashboardCardLayout	Layout persistido de um card do painel (RelX/RelY/RelWidth/RelHeight normalizados 0..1, IsVisible, ZIndex).
LogEntry	Linha imutável do console de eventos (Timestamp, Level, Message).
ArduinoCliInfo	/ Resultado da detecção/compilação do arduino-cli.exe e linhas de console associadas; ArduinoBoardCatalog.DefaultBoards é a lista curada estática de placas suportadas.
ArduinoCompileResult	
/ ArduinoCliOutputLine	
ArduinoBoardOption	

2.2 Services (regras de negócio)

Seguem o padrão interface (I*Service) + implementação, permitindo injeção de dependência e substituição em testes.

Tabela 2: Classes de serviço (Services)

Classe	Responsabilidade
SerialProtocolParser	Interpreta/monta o protocolo textual PC↔Arduino (TryParse, BuildSystemOn/Off, BuildModeDetection/Auto, BuildFire).
SerialCommunicationService (ISerialCommunicationService)	Transporte serial: listar portas, conectar, ler em loop, enviar, watchdog; depende de SerialProtocolParser e AppConfig.
TargetTrackingService (ITargetTrackingService)	Cria/atualiza/expira Target a partir de SensorReading; depende de QuadrantHelper e CoordinateConverter.
TowerSelectionService (ITowerSelectionService)	Algoritmo de seleção de torre; depende de DistanceCalculator e IDeadZoneService.
FireControlService (IFireControlService)	Regra de segurança + acionamento demonstrativo; depende de IDeadZoneService.
LoggingService (ILoggingService)	Console de eventos thread-safe (Info, Success, Warning, Error).
AuthService (IAuthService)	Login/logout/sessão; depende de IUsuarioRepository e IPasswordHasher.
PasswordHasher (IPasswordHasher)	Hash PBKDF2-HMACSHA256 de senha.
PermissionService (IPermissionService)	Regras de acesso por perfil (PodeVerMenu, PodeExecutarAcoes, PodeGerenciarUsuarios, PodeGerenciarZonasMortas).
DeadZoneService (IDeadZoneService)	Avalia se um alvo está bloqueado por uma zona morta; depende de IDeadZoneRepository.
ArduinoCliLocatorService (IArduinoCliLocatorService)	Localiza arduino-cli.exe e lista placas instaladas.

Tabela 2 (continuação)

Classe	Responsabilidade
ArduinoCompilerService (IArduinoCompilerService)	Compila o sketch como processo filho (CompileAsync).
ArduinoSettingsRepository (IArduinoSettingsRepository)	Persiste ArduinoCliSettings em JSON.
DashboardLayoutRepository (IDashboardLayoutRepository)	Persiste layout dos cards do painel em JSON.
ObjetoDetectadoExportService (IObjetoDetectadoExportService)	Exporta/importa ObjetoDetectado (CSV/XML/PDF); depende de CsvTableStore, XmlSerializer e PdfSharp.
LocalizationService (ILocalizationService)	Internacionalização pt-BR/en-US.
ThemeService (IThemeService)	Tema claro/escuro/sistema.
NavigationService (INavigationService)	Navegação entre telas da Shell.

2.3 Repositories (persistência CSV)

Tabela 3: Classes de persistência (Repositories)

Classe	Responsabilidade
CsvUsuarioRepository (IUsuarioRepository)	CRUD de Usuario em usuarios.csv.
CsvObjetoDetectadoRepository (IObjetoDetectadoRepository)	Insere/lista ObjetoDetectado em objetos_detectados.csv; expõe BuildColumns() estático reaproveitado pela exportação.
CsvAcaoRealizadaRepository (IAcaoRealizadaRepository)	Só-inserção de AcaoRealizada.
CsvAlteracaoModoRepository (IAAlteracaoModoRepository)	Só-inserção de AlteracaoModo.
CsvPreferenciasUsuarioRepository (IPreferenciasUsuarioRepository)	CRUD 1:1 de PreferenciasUsuario.
CsvChamadoAjudaRepository (IChamadoAjudaRepository)	CRUD (com Update) de ChamadoAjuda.
CsvTableStore<T>	Motor genérico de leitura/escrita CSV usado por composição em todos os repositórios acima (EnsureFileWithHeader, Read, Append).

Todos os repositórios CSV dependem de CsvTableStore<T> (composição) e de AppDataPaths (caminho em %AppData%\RadarTorres\Data\).

2.4 ViewModels

Tabela 4: Classes de apresentação (ViewModels)

Classe	Responsabilidade
ViewModelBase	Implementa INotifyPropertyChanged (SetProperty) — classe base abstrata de todas as ViewModels.

Tabela 4 (continuação)

Classe	Responsabilidade
MainViewModel	Orquestra a tela de Monitoramento (radar, serial, torres, acionamento); depende de ISerialCommunicationService, ITargetTrackingService, ITowerSelectionService, IFireControlService, ILoggingService.
ArduinoSettingsViewModel	Orquestra a aba Configurações do Arduino; depende de IArduinoCliLocatorService, IArduinoCompilerService, IArduinoSettingsRepository e da mesma instância <i>singleton</i> de ISerialCommunicationService usada por MainViewModel.
ObjetosDetectadosViewModel	Lista, exporta e importa ObjetoDetectado; depende de IObjetoDetectadoRepository, IObjetoDetectadoExportService, ILoggingService, IAuthService, IPermissionService.
PainelPrincipalViewModel	Comando de restaurar o layout padrão do painel.
LoginViewModel	Autenticação; depende de IAuthService.
ProfileViewModel	Perfil do usuário logado e troca de senha; depende de IAuthService.
HelpDeskFormViewModel	Envio de ChamadoAjuda; depende de IChamadoAjudaRepository e IAuthService.
ShellViewModel	Estado da casca da aplicação (barra lateral, navegação); depende de INavigationService e IPermissionService.
SidebarMenuEntry	Item de menu da barra lateral (classe de dado, não orquestradora).

2.5 Helpers (funções puras, sem estado)

Tabela 5: Classes utilitárias (Helpers)

Classe	Responsabilidade
CoordinateConverter	Conversão polar↔cartesiano↔tela (static).
DistanceCalculator	Distância euclidiana entre pontos (static).
QuadrantHelper	Determina o quadrante Q1–Q4 de uma coordenada (static).
RelayCommand	Implementação de ICommand (padrão MVVM manual).

3 Diagrama de Classes

O diagrama a seguir consolida as camadas de *Models*, *Services*, *Repositories* e *ViewModels* descritas acima, com os relacionamentos de implementação de interface (<|..), herança (<|--), composição, dependência e associação. Fonte: Classes_RadarTorres.puml, nesta mesma pasta.

4 Relacionamentos Principais

- Cada serviço de negócio (SerialCommunicationService, TargetTrackingService, TowerSelectionService, FireControlService, AuthService, PermissionService, LoggingService, DeadZoneService, ObjetoDetectadoExportService) implementa uma interface I*Service correspondente, permitindo injeção de dependência e substituição em testes.

- Cada repositório CSV (CsvUsuarioRepository, CsvObjetoDetectadoRepository, CsvAcaoRealizadaRepository, CsvAlteracaoModoRepository, CsvChamadoAjudaRepository) implementa a interface I*Repository correspondente e compõe CsvTableStore<T> para leitura/escrita.
- TargetTrackingService consome SensorReading e gerencia o ciclo de vida (1..*) dos objetos Target, usando QuadrantHelper e CoordinateConverter.
- TowerSelectionService gerencia (1..*) os objetos Tower e consulta IDeadZoneService antes de selecionar uma torre para um Target (Target.SelectedTower, associação 0..1-1).
- FireControlService consulta IDeadZoneService para autorizar o acionamento demonstrativo.
- Todas as ViewModels herdam de ViewModelBase (INotifyPropertyChanged); MainViewModel e ArduinoSettingsViewModel compartilham a mesma instância *singleton* de ISerialCommunicationService.
- Usuario tem relação 1:0..1 com PreferenciasUsuario e 1:* com AcaoRealizada, AlteracaoModo e ChamadoAjuda.

5 Conclusão

O diagrama de classes evidencia o uso consistente de Programação Orientada a Objetos no RadarTorres: separação entre interface e implementação em toda a camada de serviços e repositórios, composição do motor genérico CsvTableStore<T> para persistência, herança comum via ViewModelBase/INotifyPropertyChanged na camada de apresentação, e funções utilitárias sem estado isoladas em *Helpers*. Essa estrutura mantém as responsabilidades de domínio, negócio, persistência e apresentação desacopladas, favorecendo testabilidade e substituição de implementações sem alterar os consumidores.

6 Validação e Limitações

Todas as classes, atributos e métodos apresentados foram lidos diretamente dos arquivos-fonte em src/RadarTorres.App/; nenhum nome foi inventado. As responsabilidades de ILocalizationService, IThemeService, INavigationService e IDeadZoneService são, em parte, **inferidas** do uso descrito em Docs/Tecnica/ARQUITETURA.md e do nome da interface, já que a assinatura completa desses membros não foi lida nesta varredura.